

Elementary Graph Algorithms II

Dr. Bugra Caskurlu¹

¹The School of Computing
New Uzbekistan University

November, 2025

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.
- This process continues until we have discovered all the vertices that are reachable from the original source vertex.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.
- This process continues until we have discovered all the vertices that are reachable from the original source vertex.
- If any undiscovered vertices remain, then depth-first search selects one of them as a new source, and it repeats the search from that source.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.
- This process continues until we have discovered all the vertices that are reachable from the original source vertex.
- If any undiscovered vertices remain, then depth-first search selects one of them as a new source, and it repeats the search from that source.
- The algorithm repeats this entire process until it has discovered every vertex.

Basic Idea of the DFS Algorithm

A High-Level Description of DFS

- The strategy followed by depth-first search (DFS) algorithm is to search "deeper" in the graph whenever possible.
- DFS explores edges out of the most recently discovered vertex v that still has unexplored edges leaving it.
- Once all of v 's edges have been explored, the search "backtracks" to explore edges leaving the vertex from which v was discovered.
- This process continues until we have discovered all the vertices that are reachable from the original source vertex.
- If any undiscovered vertices remain, then depth-first search selects one of them as a new source, and it repeats the search from that source.
- The algorithm repeats this entire process until it has discovered every vertex.
- The algorithm works on both directed and undirected graphs.

The Predecessor Subgraph

Depth-First Forest

- As in BFS, whenever DFS discovers a vertex v during a scan of the adjacency list of an already discovered vertex u , it records this event by setting v 's predecessor $v.\pi$ to u .

The Predecessor Subgraph

Depth-First Forest

- As in BFS, whenever DFS discovers a vertex v during a scan of the adjacency list of an already discovered vertex u , it records this event by setting v 's predecessor $v.\pi$ to u .
- Unlike BFS, whose predecessor subgraph forms a tree, the predecessor subgraph produced by DFS may be composed of several trees, because the search may repeat from multiple sources.

The Predecessor Subgraph

Depth-First Forest

- As in BFS, whenever DFS discovers a vertex v during a scan of the adjacency list of an already discovered vertex u , it records this event by setting v 's predecessor $v.\pi$ to u .
- Unlike BFS, whose predecessor subgraph forms a tree, the predecessor subgraph produced by DFS may be composed of several trees, because the search may repeat from multiple sources.
- We define the predecessor subgraph of DFS as follows: Let $G_\pi = (V, E_\pi)$, where

$$E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}.$$

The Predecessor Subgraph

Depth-First Forest

- As in BFS, whenever DFS discovers a vertex v during a scan of the adjacency list of an already discovered vertex u , it records this event by setting v 's predecessor $v.\pi$ to u .
- Unlike BFS, whose predecessor subgraph forms a tree, the predecessor subgraph produced by DFS may be composed of several trees, because the search may repeat from multiple sources.
- We define the predecessor subgraph of DFS as follows: Let $G_\pi = (V, E_\pi)$, where

$$E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}.$$

- The predecessor subgraph of a DFS forms a **depth-first forest** comprising several **depth-first trees**. The edges in E_π are called **tree edges**.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.
- DFS also timestamps each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v).

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.
- DFS also timestamps each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v).
- These timestamps provide important information about the structure of the graph and are generally helpful in reasoning about the behavior of DFS.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.
- DFS also timestamps each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v).
- These timestamps provide important information about the structure of the graph and are generally helpful in reasoning about the behavior of DFS.
- The timestamps are integers between 1 and $2|V|$, since there is one discovery event and one finishing event for each of the $|V|$ vertices. For every vertex u , $u.d < u.f$.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.
- DFS also timestamps each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v).
- These timestamps provide important information about the structure of the graph and are generally helpful in reasoning about the behavior of DFS.
- The timestamps are integers between 1 and $2|V|$, since there is one discovery event and one finishing event for each of the $|V|$ vertices. For every vertex u , $u.d < u.f$.
- Vertex u is WHITE before time $u.d$, GRAY between time $u.d$ and $u.f$, and BLACK thereafter.

Additional Data Structures Used

Colors and Timestamps

- As in BFS, DFS colors vertices during the search to indicate their state: each vertex is initially white, is grayed when it is **discovered**, and is blackened when it is **finished**, that is, when its adjacency list has been examined completely.
- This technique guarantees that each vertex ends up in exactly one DFS tree, so that these trees are disjoint.
- DFS also timestamps each vertex. Each vertex v has two timestamps: the first timestamp $v.d$ records when v is first discovered (and grayed), and the second timestamp $v.f$ records when the search finishes examining v 's adjacency list (and blackens v).
- These timestamps provide important information about the structure of the graph and are generally helpful in reasoning about the behavior of DFS.
- The timestamps are integers between 1 and $2|V|$, since there is one discovery event and one finishing event for each of the $|V|$ vertices. For every vertex u , $u.d < u.f$.
- Vertex u is WHITE before time $u.d$, GRAY between time $u.d$ and $u.f$, and BLACK thereafter.
- The variable *time* is a global variable that we use for timestamping.

Pseudocode of the DFS Algorithm

DFS(G)

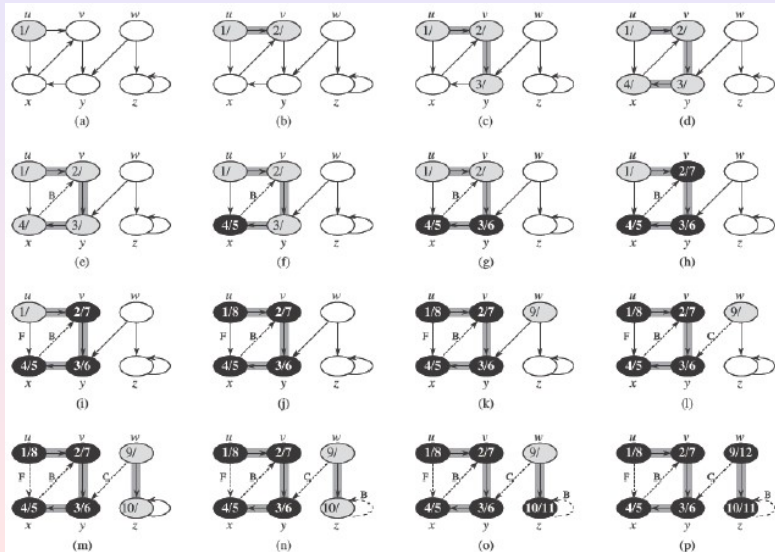
```
1  for each vertex  $u \in G.V$ 
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for each vertex  $u \in G.V$ 
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )
```

Pseudocode for the DFS-VISIT Procedure

DFS-VISIT(G, u)

```
1   $time = time + 1$                                 // white vertex  $u$  has just been discovered
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for each  $v \in G.Adj[u]$                             // explore edge  $(u, v)$ 
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$                                 // blacken  $u$ ; it is finished
9   $time = time + 1$ 
10  $u.f = time$ 
```

A Sample Run of the DFS Algorithm on a Directed Graph



Running-Time Analysis

Running-Time Analysis of DFS

- The loops on lines 1 – 3 and lines 5 – 7 of the DFS procedure take $\Theta(n)$ -time, exclusive of the time to execute the calls to the DFS-VISIT procedure.

Running-Time Analysis

Running-Time Analysis of DFS

- The loops on lines 1 – 3 and lines 5 – 7 of the DFS procedure take $\Theta(n)$ -time, exclusive of the time to execute the calls to the DFS-VISIT procedure.
- The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray.

Running-Time Analysis

Running-Time Analysis of DFS

- The loops on lines 1 – 3 and lines 5 – 7 of the DFS procedure take $\Theta(n)$ -time, exclusive of the time to execute the calls to the DFS-VISIT procedure.
- The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray.
- During an execution of the DFS-VISIT procedure on a vertex v , the loop on lines 4 – 7 executes $|Adj[v]|$ times.

Running-Time Analysis

Running-Time Analysis of DFS

- The loops on lines 1 – 3 and lines 5 – 7 of the DFS procedure take $\Theta(n)$ -time, exclusive of the time to execute the calls to the DFS-VISIT procedure.
- The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray.
- During an execution of the DFS-VISIT procedure on a vertex v , the loop on lines 4 – 7 executes $|Adj[v]|$ times.
- Since

$$\sum_{v \in V} |Adj[v]| = m$$

the total cost of executing DFS-VISIT is $\Theta(n + m)$.

Running-Time Analysis

Running-Time Analysis of DFS

- The loops on lines 1 – 3 and lines 5 – 7 of the DFS procedure take $\Theta(n)$ -time, exclusive of the time to execute the calls to the DFS-VISIT procedure.
- The procedure DFS-VISIT is called exactly once for each vertex $v \in V$, since the vertex u on which DFS-VISIT is invoked must be white and the first thing DFS-VISIT does is paint vertex u gray.
- During an execution of the DFS-VISIT procedure on a vertex v , the loop on lines 4 – 7 executes $|Adj[v]|$ times.
- Since

$$\sum_{v \in V} |Adj[v]| = m$$

the total cost of executing DFS-VISIT is $\Theta(n + m)$.

- The total running time of DFS is therefore $\Theta(n + m)$.

The Structure Of The Predecessor Subgraph

The Predecessor Subgraph is a Forest

- DFS yields valuable information about the structure of a graph.

The Structure Of The Predecessor Subgraph

The Predecessor Subgraph is a Forest

- DFS yields valuable information about the structure of a graph.
- The most basic property of DFS is that the predecessor subgraph G_π does indeed form a forest of trees, since the structure of the depth-first trees exactly mirrors the structure of recursive calls of DFS-VISIT.

The Structure Of The Predecessor Subgraph

The Predecessor Subgraph is a Forest

- DFS yields valuable information about the structure of a graph.
- The most basic property of DFS is that the predecessor subgraph G_π does indeed form a forest of trees, since the structure of the depth-first trees exactly mirrors the structure of recursive calls of DFS-VISIT.
- That is, $u = v.\pi$ if and only if DFS-VISIT on v was called during a search of u 's adjacency list.

The Structure Of The Predecessor Subgraph

The Predecessor Subgraph is a Forest

- DFS yields valuable information about the structure of a graph.
- The most basic property of DFS is that the predecessor subgraph G_π does indeed form a forest of trees, since the structure of the depth-first trees exactly mirrors the structure of recursive calls of DFS-VISIT.
- That is, $u = v.\pi$ if and only if DFS-VISIT on v was called during a search of u 's adjacency list.
- Therefore, there cannot be a cycle in the predecessor subgraph G_π .

The Structure Of The Predecessor Subgraph

The Predecessor Subgraph is a Forest

- DFS yields valuable information about the structure of a graph.
- The most basic property of DFS is that the predecessor subgraph G_π does indeed form a forest of trees, since the structure of the depth-first trees exactly mirrors the structure of recursive calls of DFS-VISIT.
- That is, $u = v.\pi$ if and only if DFS-VISIT on v was called during a search of u 's adjacency list.
- Therefore, there cannot be a cycle in the predecessor subgraph G_π .
- Additionally, vertex v is a descendant of vertex u in the depth-first forest if and only if v is discovered during the time in which u is gray.

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- 1 the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- 1 the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- 2 the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- ❶ the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- ❷ the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- ❸ the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- ❶ the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- ❷ the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- ❸ the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Proof

- Assume that $u.d < v.d$.

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- ❶ the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- ❷ the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- ❸ the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Proof

- Assume that $u.d < v.d$.

- ❶ If $v.d < u.f$, v was discovered while u was still gray, which implies that v is a descendant of u . Moreover, since v was discovered more recently than u , all of its outgoing edges are explored, and v is finished, before the search returns to and finishes u . In this case, $[v.d, v.f]$ is entirely contained within $[u.d, u.f]$.

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- ❶ the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- ❷ the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- ❸ the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Proof

- Assume that $u.d < v.d$.

- ❶ If $v.d < u.f$, v was discovered while u was still gray, which implies that v is a descendant of u . Moreover, since v was discovered more recently than u , all of its outgoing edges are explored, and v is finished, before the search returns to and finishes u . In this case, $[v.d, v.f]$ is entirely contained within $[u.d, u.f]$.
- ❷ If $u.f < v.d$, then $u.d < u.f < v.d < v.f$, and the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint. But then neither vertex was discovered while the other was gray, and so neither vertex is a descendant of the other.

The Paranthesis Structure Of Discovery and Finishing Times

The Paranthesis Theorem

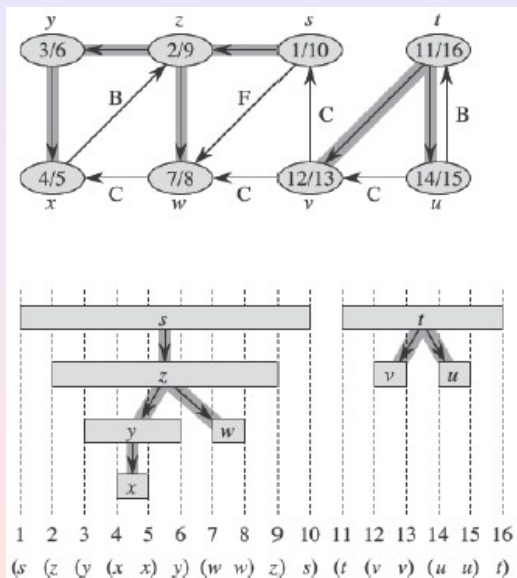
In any depth-first search of a (directed or undirected) graph $G = (V, E)$, for any two vertices u and v , exactly one of the following three conditions holds:

- 1 the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,
- 2 the interval $[u.d, u.f]$ is contained entirely within the interval $[v.d, v.f]$, and u is a descendant of v in a depth-first tree, or
- 3 the interval $[v.d, v.f]$ is contained entirely within the interval $[u.d, u.f]$, and v is a descendant of u in a depth-first tree.

Proof

- Assume that $u.d < v.d$.
 - 1 If $v.d < u.f$, v was discovered while u was still gray, which implies that v is a descendant of u . Moreover, since v was discovered more recently than u , all of its outgoing edges are explored, and v is finished, before the search returns to and finishes u . In this case, $[v.d, v.f]$ is entirely contained within $[u.d, u.f]$.
 - 2 If $u.f < v.d$, then $u.d < u.f < v.d < v.f$, and the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint. But then neither vertex was discovered while the other was gray, and so neither vertex is a descendant of the other.
- $v.d < u.d$ case is similar, with the roles of u and v reversed in the above argument.

A Sample Run of DFS Where We Keep Track Of Timestamps



A Corollary Of The Paranthesis Theorem and the White Path Theorem

Corollary

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

A Corollary Of The Paranthesis Theorem and the White Path Theorem

Corollary

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

White Path Theorem

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

A Corollary Of The Paranthesis Theorem and the White Path Theorem

Corollary

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

White Path Theorem

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

Proof Idea

- Consider the moment $u.d$ that vertex u is discovered.

A Corollary Of The Paranthesis Theorem and the White Path Theorem

Corollary

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

White Path Theorem

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

Proof Idea

- Consider the moment $u.d$ that vertex u is discovered.
- The DFS-VISIT from u will discover all the WHITE vertices that are reachable from u at the moment, and all such vertices will be descendants of u in the depth-first forest.

A Corollary Of The Paranthesis Theorem and the White Path Theorem

Corollary

Vertex v is a proper descendant of vertex u in the depth-first forest for a (directed or undirected) graph G if and only if $u.d < v.d < v.f < u.f$.

White Path Theorem

In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u , there is a path from u to v consisting entirely of white vertices.

Proof Idea

- Consider the moment $u.d$ that vertex u is discovered.
- The DFS-VISIT from u will discover all the WHITE vertices that are reachable from u at the moment, and all such vertices will be descendants of u in the depth-first forest.
- The DFS-VISIT from u cannot discover any other vertices and they will not be descendants of u in the depth-first forest.

Classification of Edges

Not All Nontree Edges Are The Same

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

Classification of Edges

Not All Nontree Edges Are The Same

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

- 1 **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .

Classification of Edges

Not All Nontree Edges Are The Same

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

- 1 **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
- 2 **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.

Classification of Edges

Not All Nontree Edges Are The Same

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

- 1 **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
- 2 **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
- 3 **Forward edges** are those *nontree* edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.

Classification of Edges

Not All Nontree Edges Are The Same

We can define four edge types in terms of the depth-first forest G_π produced by a depth-first search on G :

- 1 **Tree edges** are edges in the depth-first forest G_π . Edge (u, v) is a tree edge if v was first discovered by exploring edge (u, v) .
- 2 **Back edges** are those edges (u, v) connecting a vertex u to an ancestor v in a depth-first tree. We consider self-loops, which may occur in directed graphs, to be back edges.
- 3 **Forward edges** are those *nontree* edges (u, v) connecting a vertex u to a descendant v in a depth-first tree.
- 4 **Cross edges** are all other edges. They can go between vertices in the same depth-first tree, as long as one vertex is not an ancestor of the other, or they can go between vertices in different depth-first trees.

Classifying Edges During The Execution Of DFS

Node Colors Convey Information About Edge Types

The DFS algorithm has enough information to classify edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

Classifying Edges During The Execution Of DFS

Node Colors Convey Information About Edge Types

The DFS algorithm has enough information to classify edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

- 1 WHITE indicates a tree edge. This is immediate from the specification of the algorithm.

Classifying Edges During The Execution Of DFS

Node Colors Convey Information About Edge Types

The DFS algorithm has enough information to classify edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

- 1 WHITE indicates a tree edge. This is immediate from the specification of the algorithm.
- 2 GRAY indicates a back edge. This is because gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations. Notice that the number of gray vertices is one more than the depth in the depth-first forest of the vertex most recently discovered. Exploration always proceeds from the deepest gray vertex, so an edge that reaches another gray vertex has reached an ancestor.

Classifying Edges During The Execution Of DFS

Node Colors Convey Information About Edge Types

The DFS algorithm has enough information to classify edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

- 1 WHITE indicates a tree edge. This is immediate from the specification of the algorithm.
- 2 GRAY indicates a back edge. This is because gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations. Notice that the number of gray vertices is one more than the depth in the depth-first forest of the vertex most recently discovered. Exploration always proceeds from the deepest gray vertex, so an edge that reaches another gray vertex has reached an ancestor.
- 3 BLACK indicates a forward or cross edge. To determine whether it is a forward or a cross edges, we need to check the discovery times of u and v .

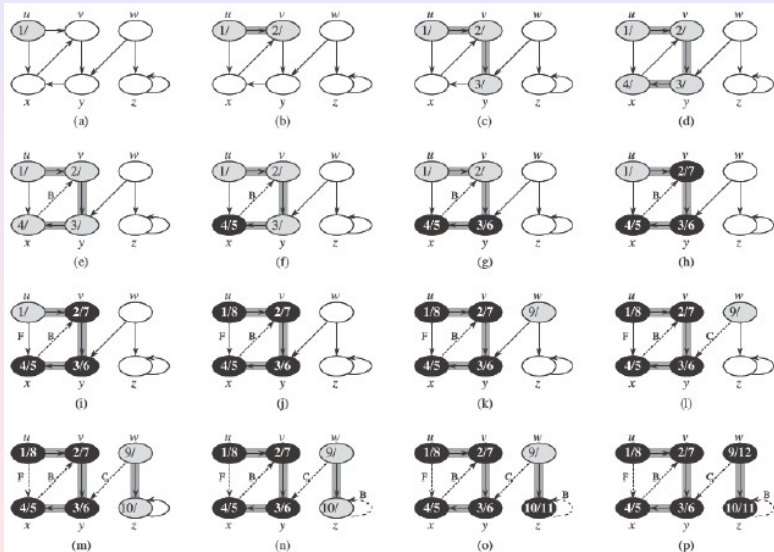
Classifying Edges During The Execution Of DFS

Node Colors Convey Information About Edge Types

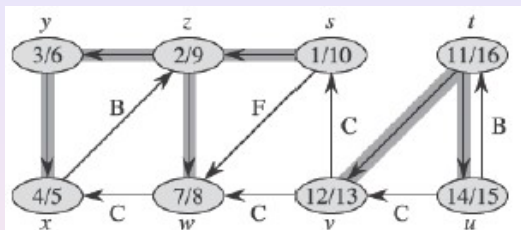
The DFS algorithm has enough information to classify edges as it encounters them. The key idea is that when we first explore an edge (u, v) , the color of vertex v tells us something about the edge:

- ❶ WHITE indicates a tree edge. This is immediate from the specification of the algorithm.
- ❷ GRAY indicates a back edge. This is because gray vertices always form a linear chain of descendants corresponding to the stack of active DFS-VISIT invocations. Notice that the number of gray vertices is one more than the depth in the depth-first forest of the vertex most recently discovered. Exploration always proceeds from the deepest gray vertex, so an edge that reaches another gray vertex has reached an ancestor.
- ❸ BLACK indicates a forward or cross edge. To determine whether it is a forward or a cross edges, we need to check the discovery times of u and v .
 - If $u.d < v.d$, then (u, v) is a forward edge.
 - If $u.d > v.d$, then (u, v) is a cross edge.

Edges Classified While Running DFS Algorithm on a Directed Graph



A Sample Run of the DFS Algorithm on Another Directed Graph



A Sample Run of the DFS Algorithm on Another Directed Graph

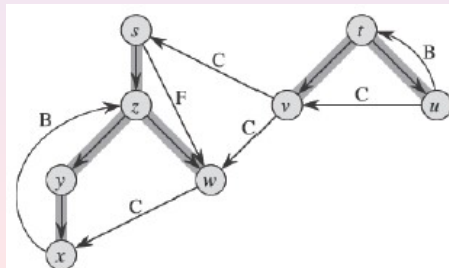
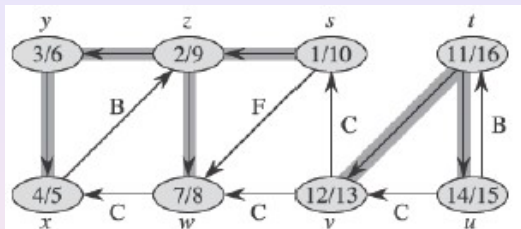


Figure: Same graph redrawn with all tree and forward edges going down and back edges going up.

Classification of Edges for Undirected Graphs

Resolving Ambiguity

- An undirected graph may entail some ambiguity in how we classify edges, since (u, v) and (v, u) are really the same edge.

Classification of Edges for Undirected Graphs

Resolving Ambiguity

- An undirected graph may entail some ambiguity in how we classify edges, since (u, v) and (v, u) are really the same edge.
- In such a case, we classify the edge according to whichever of (u, v) or (v, u) the search encounters first.

Classification of Edges for Undirected Graphs

Resolving Ambiguity

- An undirected graph may entail some ambiguity in how we classify edges, since (u, v) and (v, u) are really the same edge.
- In such a case, we classify the edge according to whichever of (u, v) or (v, u) the search encounters first.
- That is equivalent to saying that we classify the edge as the *first* type in the classification list (Tree, Back, Forward, Cross) that applies.

Classification of Edges for Undirected Graphs

Resolving Ambiguity

- An undirected graph may entail some ambiguity in how we classify edges, since (u, v) and (v, u) are really the same edge.
- In such a case, we classify the edge according to whichever of (u, v) or (v, u) the search encounters first.
- That is equivalent to saying that we classify the edge as the *first* type in the classification list (Tree, Back, Forward, Cross) that applies.
- Notice that an edge between a node v and her predecessor $v.\pi$ is classified as tree edge, not a back edge.

Classification of Edges for Undirected Graphs

Resolving Ambiguity

- An undirected graph may entail some ambiguity in how we classify edges, since (u, v) and (v, u) are really the same edge.
- In such a case, we classify the edge according to whichever of (u, v) or (v, u) the search encounters first.
- That is equivalent to saying that we classify the edge as the *first* type in the classification list (Tree, Back, Forward, Cross) that applies.
- Notice that an edge between a node v and her predecessor $v.\pi$ is classified as tree edge, not a back edge.
- Similarly, a nontree edge between a node v and her descendant is classified as back edge, not a forward edge.

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof

- Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$.

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof

- Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$.
- Then the search must discover and finish v before it finishes u (while u is gray), since v is on u 's adjacency list.

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof

- Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$.
- Then the search must discover and finish v before it finishes u (while u is gray), since v is on u 's adjacency list.
- If the first time that the search explores edge (u, v) , it is in the direction from u to v , then v is undiscovered (white) until that time, for otherwise the search would have explored this edge already in the direction from v to u .

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof

- Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$.
- Then the search must discover and finish v before it finishes u (while u is gray), since v is on u 's adjacency list.
- If the first time that the search explores edge (u, v) , it is in the direction from u to v , then v is undiscovered (white) until that time, for otherwise the search would have explored this edge already in the direction from v to u .
- Thus, (u, v) becomes a tree edge.

Fewer Edge Types in Undirected Graphs

Theorem

In a depth-first search of an undirected graph G , every edge of G is either a tree edge or a back edge.

Proof

- Let (u, v) be an arbitrary edge of G , and suppose without loss of generality that $u.d < v.d$.
- Then the search must discover and finish v before it finishes u (while u is gray), since v is on u 's adjacency list.
- If the first time that the search explores edge (u, v) , it is in the direction from u to v , then v is undiscovered (white) until that time, for otherwise the search would have explored this edge already in the direction from v to u .
- Thus, (u, v) becomes a tree edge.
- If the search explores (u, v) first in the direction from v to u , then (u, v) is a back edge, since u is still gray at the time the edge is first explored.

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Proof

- Let (u, v) be an arbitrary edge of G . Notice that u and v are in the same BFS tree since each BFS tree correspond to a connected component (CC) of the graph, and u and v are in the same CC.

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Proof

- Let (u, v) be an arbitrary edge of G . Notice that u and v are in the same BFS tree since each BFS tree correspond to a connected component (CC) of the graph, and u and v are in the same CC.
- If (u, v) is a back/forward edge then one of them is an ancestor of the other. Assume that u is an ancestor of v .

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Proof

- Let (u, v) be an arbitrary edge of G . Notice that u and v are in the same BFS tree since each BFS tree correspond to a connected component (CC) of the graph, and u and v are in the same CC.
- If (u, v) is a back/forward edge then one of them is an ancestor of the other. Assume that u is an ancestor of v .
- u is not the predecessor of v since otherwise (u, v) would be a tree edge.

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Proof

- Let (u, v) be an arbitrary edge of G . Notice that u and v are in the same BFS tree since each BFS tree correspond to a connected component (CC) of the graph, and u and v are in the same CC.
- If (u, v) is a back/forward edge then one of them is an ancestor of the other. Assume that u is an ancestor of v .
- u is not the predecessor of v since otherwise (u, v) would be a tree edge.
- Then, v is at least 2 levels below u in the breadth-first tree. That cannot be true since u would discover v while her adjacency list is examined.

Edge Types of Undirected Graphs For BFS

Theorem

In a breadth-first search of an undirected graph G (assume that the search is repeated from an undiscovered vertex if all vertices are not explored), every edge of G is either a tree edge or a cross edge.

Furthermore, for a cross edge (u, v) , we have $|u.d - v.d| \leq 1$, where $u.d$ and $v.d$ refer to the distances of u and v from the source node s respectively.

Proof

- Let (u, v) be an arbitrary edge of G . Notice that u and v are in the same BFS tree since each BFS tree correspond to a connected component (CC) of the graph, and u and v are in the same CC.
- If (u, v) is a back/forward edge then one of them is an ancestor of the other. Assume that u is an ancestor of v .
- u is not the predecessor of v since otherwise (u, v) would be a tree edge.
- Then, v is at least 2 levels below u in the breadth-first tree. That cannot be true since u would discover v while her adjacency list is examined.
- Thus, (u, v) is not a back edge or a forward edge. If it is a cross edge, then $|u.d - v.d| \leq 1$ since otherwise whichever u and v is discovered first would discover the other. In this case, (u, v) would be a tree edge.

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$,

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .
- If $v.d \leq u.d < u.f \leq v.f$,

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .
- If $v.d \leq u.d < u.f \leq v.f$, then (u, v) is a back edge. This is because u is a descendant of v .

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .
- If $v.d \leq u.d < u.f \leq v.f$, then (u, v) is a back edge. This is because u is a descendant of v .
- If $v.d < v.f < u.d < u.f$,

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .
- If $v.d \leq u.d < u.f \leq v.f$, then (u, v) is a back edge. This is because u is a descendant of v .
- If $v.d < v.f < u.d < u.f$, then (u, v) is a cross edge. This is because v is finished before u is discovered.

Question 1

Determining Edge Types By Looking At Discovery And Finishing Types

Let (u, v) be an edge of a directed graph $G = (V, E)$. Assume that a depth-first forest is obtained by an executing of the DFS algorithm. What can you say about the type of (u, v) in each of the following situations:

- If $u.d < v.d < v.f < u.f$, then (u, v) is either a tree edge or a forward edge. This is because u is an ancestor of v .
- If $v.d \leq u.d < u.f \leq v.f$, then (u, v) is a back edge. This is because u is a descendant of v .
- If $v.d < v.f < u.d < u.f$, then (u, v) is a cross edge. This is because v is finished before u is discovered.

Converse of the above statements are also correct.

Iterative Implementation of DFS Using A Stack

```
DFS-STACK(G)
  for each vertex  $u \in G.V$ 
     $u.color = WHITE$ 
     $u.\pi = NIL$ 
  time = 0
  for each vertex  $u \in G.V$ 
    if  $u.color == WHITE$ 
      DFS-VISIT-STACK(G, u)
```

Pseudocode For DFS-VISIT-STACK

```
DFS-VISIT-STACK(G, u)
  S =  $\emptyset$ 
  PUSH(S, u)
  time = time + 1           // white vertex u has just been discovered
  u.d = time
  u.color = GRAY
  while !STACK-EMPTY(S)
    u = TOP(S)
    v = FIRST-WHITE-NEIGHBOR(G, u)
    if v == NIL
      // u's adjacency list has been fully explored
      POP(S)
      time = time + 1
      u.f = time
      u.color = BLACK       // blackend u; it is finished
    else
      // u's adjacency list hasn't been fully explored
      v. $\pi$  = u
      time = time + 1
      v.d = time
      v.color = GRAY
      PUSH(S, v)
```

Pseudocode For FIRST-WHITE-NEIGHBOR

```
FIRST-WHITE-NEIGHBOR(G, u)
  for each vertex  $v \in G.\text{Adj}[u]$ 
    if  $v.\text{color} == \text{WHITE}$ 
      return  $v$ 
  return NIL
```

Pseudocode For FIRST-WHITE-NEIGHBOR

```
FIRST-WHITE-NEIGHBOR(G, u)
  for each vertex  $v \in G.\text{Adj}[u]$ 
    if  $v.\text{color} == \text{WHITE}$ 
      return  $v$ 
  return NIL
```

Running Time

- What is the running time of this iterative implementation of DFS?

Pseudocode For FIRST-WHITE-NEIGHBOR

```
FIRST-WHITE-NEIGHBOR( $G, u$ )  
  for each vertex  $v \in G.Adj[u]$   
    if  $v.color == WHITE$   
      return  $v$   
  return NIL
```

Running Time

- What is the running time of this iterative implementation of DFS?
- **Answer:** For a node u , FIRST-WHITE-NEIGHBOR procedure can be called on u more than once. In fact, it may be called as many as $|Adj[u]| < n$. Each such call will take $O(n)$ time on average. Thus, the running time can be as high as $O(n^3)$.

Pseudocode For FIRST-WHITE-NEIGHBOR

```
FIRST-WHITE-NEIGHBOR( $G, u$ )  
  for each vertex  $v \in G.Adj[u]$   
    if  $v.color == WHITE$   
      return  $v$   
  return NIL
```

Running Time

- What is the running time of this iterative implementation of DFS?
- **Answer:** For a node u , FIRST-WHITE-NEIGHBOR procedure can be called on u more than once. In fact, it may be called as many as $|Adj[u]| < n$. Each such call will take $O(n)$ time on average. Thus, the running time can be as high as $O(n^3)$.
- **Exercise:** You will come up with an iterative implementation that runs in $O(n + m)$ -time in the afternoon.

Pseudocode For FIRST-WHITE-NEIGHBOR

```
FIRST-WHITE-NEIGHBOR(G, u)
  for each vertex  $v \in G.Adj[u]$ 
    if  $v.color == WHITE$ 
      return  $v$ 
  return NIL
```

Running Time

- What is the running time of this iterative implementation of DFS?
- **Answer:** For a node u , FIRST-WHITE-NEIGHBOR procedure can be called on u more than once. In fact, it may be called as many as $|Adj[u]| < n$. Each such call will take $O(n)$ time on average. Thus, the running time can be as high as $O(n^3)$.
- **Exercise:** You will come up with an iterative implementation that runs in $O(n + m)$ -time in the afternoon.
- **Hint:** When processing a node, try putting all its white neighbors to the stack. The predecessor of a node is the one who pushed it to the stack for the last time. Notice that a node can be pushed into the stack multiple times. In fact, it may be pushed as much as the number of incoming edges it has.

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, we next discover u .

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, we next discover u .
- The only adjacent vertex to u is w , but w is already grey, so u finishes.

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, we next discover u .
- The only adjacent vertex to u is w , but w is already grey, so u finishes.
- Since v is not yet a descendant of u and u is finished, v can never be a descendant of u .

Question 2

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , and if $u.d < v.d$ in a depth-first search of G , then v is a descendant of u in the depth-first forest produced.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, we next discover u .
- The only adjacent vertex to u is w , but w is already grey, so u finishes.
- Since v is not yet a descendant of u and u is finished, v can never be a descendant of u .
- **Notice:** Existence of a path from u to v is not sufficient for v to be a descendant of u , as we see in this example. However, due to *White Path Theorem*, if there is a path from u to v consisting of entirely white nodes at the time $u.d$, then v will be a descendant of u .

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Notice that there is a path from u to v that passes through w .

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Notice that there is a path from u to v that passes through w .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, $w.d = 1$, and $u.d = 2$.

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Notice that there is a path from u to v that passes through w .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, $w.d = 1$, and $u.d = 2$.
- The only adjacent vertex to u is w , but w is already grey, so u finishes. Thus, $u.f = 3$.

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Notice that there is a path from u to v that passes through w .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, $w.d = 1$, and $u.d = 2$.
- The only adjacent vertex to u is w , but w is already grey, so u finishes. Thus, $u.f = 3$.
- The search backtracks to w , and continues with discovering v . Thus, $v.d = 4$.

Question 3

Disproving A Claim

Disprove the following false assertion by presenting a counterexample:

If a directed graph G contains a path from u to v , then any depth-first search must result in $v.d \leq u.f$.

- Consider a graph with 3 vertices u , v , and w , and with edges (w, u) , (u, w) , and (w, v) .
- Notice that there is a path from u to v that passes through w .
- Suppose that DFS first explores w , and that w 's adjacency list has u before v . Thus, $w.d = 1$, and $u.d = 2$.
- The only adjacent vertex to u is w , but w is already grey, so u finishes. Thus, $u.f = 3$.
- The search backtracks to w , and continues with discovering v . Thus, $v.d = 4$.
- Notice that $v.d > u.f$.

Question 4

My Own Tree

Explain how a vertex v of a directed graph can end up in a depth-first tree containing only v , even though v has both incoming and outgoing edges in G .

Question 4

My Own Tree

Explain how a vertex v of a directed graph can end up in a depth-first tree containing only v , even though v has both incoming and outgoing edges in G .

- Suppose that we have a directed graph with 3 vertices u , v , and w , and with edges (u, v) and (v, w) .

Question 4

My Own Tree

Explain how a vertex v of a directed graph can end up in a depth-first tree containing only v , even though v has both incoming and outgoing edges in G .

- Suppose that we have a directed graph with 3 vertices u , v , and w , and with edges (u, v) and (v, w) .
- Notice that v has both incoming and outgoing edges.

Question 4

My Own Tree

Explain how a vertex v of a directed graph can end up in a depth-first tree containing only v , even though v has both incoming and outgoing edges in G .

- Suppose that we have a directed graph with 3 vertices u , v , and w , and with edges (u, v) and (v, w) .
- Notice that v has both incoming and outgoing edges.
- If we pick our first root to be w , w will be in its own DFS tree.

Question 4

My Own Tree

Explain how a vertex v of a directed graph can end up in a depth-first tree containing only v , even though v has both incoming and outgoing edges in G .

- Suppose that we have a directed graph with 3 vertices u , v , and w , and with edges (u, v) and (v, w) .
- Notice that v has both incoming and outgoing edges.
- If we pick our first root to be w , w will be in its own DFS tree.
- Then, we pick our second root to be v . Since the only node adjacent to it has already been marked BLACK, v won't be discovering w . Thus, v will be in its own DFS tree.

Directed Acyclic Graphs

Definition

- If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree.

Directed Acyclic Graphs

Definition

- If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree.
- But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure.

Directed Acyclic Graphs

Definition

- If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree.
- But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure.
- For example, such graphs can have a large number of edges: if we start with the node set $\{1, 2, \dots, n\}$ and include an edge (i, j) whenever $i < j$, then the resulting directed graph has $\binom{n}{2}$ edges but no cycles.

Directed Acyclic Graphs

Definition

- If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree.
- But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure.
- For example, such graphs can have a large number of edges: if we start with the node set $\{1, 2, \dots, n\}$ and include an edge (i, j) whenever $i < j$, then the resulting directed graph has $\binom{n}{2}$ edges but no cycles.
- If a directed graph has no cycles, it is called a directed acyclic graph (DAG).

Directed Acyclic Graphs

Definition

- If an undirected graph has no cycles, then it has an extremely simple structure: each of its connected components is a tree.
- But it is possible for a directed graph to have no (directed) cycles and still have a very rich structure.
- For example, such graphs can have a large number of edges: if we start with the node set $\{1, 2, \dots, n\}$ and include an edge (i, j) whenever $i < j$, then the resulting directed graph has $\binom{n}{2}$ edges but no cycles.
- If a directed graph has no cycles, it is called a directed acyclic graph (DAG).

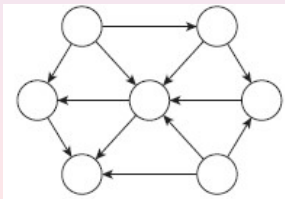


Figure: A DAG with a lot of edges.

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.
- Thus, G contains a path from v to u , and the back edge (u, v) completes a cycle.

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.
- Thus, G contains a path from v to u , and the back edge (u, v) completes a cycle.
- $\leftarrow$: Suppose that G contains a cycle c . We show that a depth-first search of G yields a back edge.

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.
- Thus, G contains a path from v to u , and the back edge (u, v) completes a cycle.
- $\leftarrow$: Suppose that G contains a cycle c . We show that a depth-first search of G yields a back edge.
- Let v be the first vertex to be discovered in c , and let (u, v) be the preceding edge in c .

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.
- Thus, G contains a path from v to u , and the back edge (u, v) completes a cycle.
- $\leftarrow$: Suppose that G contains a cycle c . We show that a depth-first search of G yields a back edge.
- Let v be the first vertex to be discovered in c , and let (u, v) be the preceding edge in c .
- At time $v.d$, the vertices of c form a path of white vertices from v to u .

Using DFS to Check Whether a Given Directed Graph is a DAG

Theorem

A directed graph G is acyclic if and only if a depth-first search of G yields no back edges.

Proof

- $\rightarrow$: Suppose that a depth-first search produces a back edge (u, v) .
- Then vertex v is an ancestor of vertex u in the depth-first forest.
- Thus, G contains a path from v to u , and the back edge (u, v) completes a cycle.
- $\leftarrow$: Suppose that G contains a cycle c . We show that a depth-first search of G yields a back edge.
- Let v be the first vertex to be discovered in c , and let (u, v) be the preceding edge in c .
- At time $v.d$, the vertices of c form a path of white vertices from v to u .
- By the white-path theorem, vertex u becomes a descendant of v in the depth-first forest. Therefore, (u, v) is a back edge.

Relating DAGs and Topological Ordering

Definition

For a directed graph G , we say that a **topological ordering** of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point "forward" in the ordering.

Relating DAGs and Topological Ordering

Definition

For a directed graph G , we say that a **topological ordering** of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point "forward" in the ordering.

Lemma

If G has a topological ordering, then G is a DAG.

Relating DAGs and Topological Ordering

Definition

For a directed graph G , we say that a **topological ordering** of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point "forward" in the ordering.

Lemma

If G has a topological ordering, then G is a DAG.

Proof

- Suppose, by way of contradiction, that G has a topological ordering $v_1, v_2, \dots, v_n$, and also has a cycle C .

Relating DAGs and Topological Ordering

Definition

For a directed graph G , we say that a **topological ordering** of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point "forward" in the ordering.

Lemma

If G has a topological ordering, then G is a DAG.

Proof

- Suppose, by way of contradiction, that G has a topological ordering $v_1, v_2, \dots, v_n$, and also has a cycle C .
- Let v_i be the lowest-indexed node on C , and let v_j be the node on C just before v_i , i.e., $(v_j, v_i) \in E$.

Relating DAGs and Topological Ordering

Definition

For a directed graph G , we say that a **topological ordering** of G is an ordering of its nodes as $v_1, v_2, \dots, v_n$ so that for every edge (v_i, v_j) , we have $i < j$. In other words, all edges point "forward" in the ordering.

Lemma

If G has a topological ordering, then G is a DAG.

Proof

- Suppose, by way of contradiction, that G has a topological ordering $v_1, v_2, \dots, v_n$, and also has a cycle C .
- Let v_i be the lowest-indexed node on C , and let v_j be the node on C just before v_i , i.e., $(v_j, v_i) \in E$.
- But by our choice of i , we have $j > i$, which contradicts the assumption that $v_1, v_2, \dots, v_n$ was a topological ordering.

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Proof

- Let G be a directed graph in which every node has at least one incoming edge. To prove the claim, we next show how to find a cycle in G .

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Proof

- Let G be a directed graph in which every node has at least one incoming edge. To prove the claim, we next show how to find a cycle in G .
- We pick any node v , and begin following edges backward from v : since v has at least one incoming edge (u, v) , we can walk backward to u ; then, since u has at least one incoming edge (x, u) , we can walk backward to x ; and so on.

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Proof

- Let G be a directed graph in which every node has at least one incoming edge. To prove the claim, we next show how to find a cycle in G .
- We pick any node v , and begin following edges backward from v : since v has at least one incoming edge (u, v) , we can walk backward to u ; then, since u has at least one incoming edge (x, u) , we can walk backward to x ; and so on.
- We can continue this process indefinitely, since every node we encounter has an incoming edge.

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Proof

- Let G be a directed graph in which every node has at least one incoming edge. To prove the claim, we next show how to find a cycle in G .
- We pick any node v , and begin following edges backward from v : since v has at least one incoming edge (u, v) , we can walk backward to u ; then, since u has at least one incoming edge (x, u) , we can walk backward to x ; and so on.
- We can continue this process indefinitely, since every node we encounter has an incoming edge.
- But after $n + 1$ steps, we will have visited some node w twice.

Indegrees of Vertices of DAGs

Lemma

In every DAG G , there is a node v with no incoming edges.

Proof

- Let G be a directed graph in which every node has at least one incoming edge. To prove the claim, we next show how to find a cycle in G .
- We pick any node v , and begin following edges backward from v : since v has at least one incoming edge (u, v) , we can walk backward to u ; then, since u has at least one incoming edge (x, u) , we can walk backward to x ; and so on.
- We can continue this process indefinitely, since every node we encounter has an incoming edge.
- But after $n + 1$ steps, we will have visited some node w twice.
- If we let C denote the sequence of nodes encountered between successive visits to w , then clearly C forms a cycle.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.
- Suppose it is true for DAGs with up to some number of nodes n .

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.
- Suppose it is true for DAGs with up to some number of nodes n .
- Given a DAG G on $n + 1$ nodes, we find a node v with no incoming edges, which is guaranteed to exist. We place v first in the topological ordering; this is safe, since all edges out of v will point forward.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.
- Suppose it is true for DAGs with up to some number of nodes n .
- Given a DAG G on $n + 1$ nodes, we find a node v with no incoming edges, which is guaranteed to exist. We place v first in the topological ordering; this is safe, since all edges out of v will point forward.
- Now $G - \{v\}$ is a DAG, since deleting v cannot create any cycles that weren't there previously.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.
- Suppose it is true for DAGs with up to some number of nodes n .
- Given a DAG G on $n + 1$ nodes, we find a node v with no incoming edges, which is guaranteed to exist. We place v first in the topological ordering; this is safe, since all edges out of v will point forward.
- Now $G - \{v\}$ is a DAG, since deleting v cannot create any cycles that weren't there previously.
- Also, $G - \{v\}$ has n nodes, so we can apply the induction hypothesis to obtain a topological ordering of $G - \{v\}$.

Every DAG has a Topological Ordering

Lemma

If G is a DAG, then G has a topological ordering.

Proof

- Let G be a DAG. We know that there exists a node v with no incoming edges.
- Let us claim by induction on the number of nodes that every DAG has a topological ordering. This is clearly true for DAGs on one or two nodes.
- Suppose it is true for DAGs with up to some number of nodes n .
- Given a DAG G on $n + 1$ nodes, we find a node v with no incoming edges, which is guaranteed to exist. We place v first in the topological ordering; this is safe, since all edges out of v will point forward.
- Now $G - \{v\}$ is a DAG, since deleting v cannot create any cycles that weren't there previously.
- Also, $G - \{v\}$ has n nodes, so we can apply the induction hypothesis to obtain a topological ordering of $G - \{v\}$.
- We append the nodes of $G - \{v\}$ in this order after v ; this is an ordering of G in which all edges point forward, and hence it is a topological ordering.

A Topological Ordering Algorithm

```
To compute a topological ordering of  $G$ :  
Find a node  $v$  with no incoming edges and order it first  
Delete  $v$  from  $G$   
Recursively compute a topological ordering of  $G - \{v\}$   
and append this order after  $v$ 
```

Figure: Converting the proof into a topological sort algorithm.

A Topological Ordering Algorithm

```
To compute a topological ordering of  $G$ :  
Find a node  $v$  with no incoming edges and order it first  
Delete  $v$  from  $G$   
Recursively compute a topological ordering of  $G - \{v\}$   
and append this order after  $v$ 
```

Figure: Converting the proof into a topological sort algorithm.

Discussing the Algorithm

- The above algorithm is directly deduced from our proof for the claim that "every DAG" has a topological ordering. A naive implementation of this algorithm will take $O(nm)$ -time.

A Topological Ordering Algorithm

```
To compute a topological ordering of  $G$ :  
Find a node  $v$  with no incoming edges and order it first  
Delete  $v$  from  $G$   
Recursively compute a topological ordering of  $G - \{v\}$   
and append this order after  $v$ 
```

Figure: Converting the proof into a topological sort algorithm.

Discussing the Algorithm

- The above algorithm is directly deduced from our proof for the claim that "every DAG" has a topological ordering. A naive implementation of this algorithm will take $O(nm)$ -time.
- **Exercise:** You will come up with an iterative implementation of this algorithm that runs in $O(n + m)$ -time in the lab session of this afternoon.

A Topological Ordering Algorithm

```
To compute a topological ordering of  $G$ :  
Find a node  $v$  with no incoming edges and order it first  
Delete  $v$  from  $G$   
Recursively compute a topological ordering of  $G - \{v\}$   
and append this order after  $v$ 
```

Figure: Converting the proof into a topological sort algorithm.

Discussing the Algorithm

- The above algorithm is directly deduced from our proof for the claim that "every DAG" has a topological ordering. A naive implementation of this algorithm will take $O(nm)$ -time.
- **Exercise:** You will come up with an iterative implementation of this algorithm that runs in $O(n + m)$ -time in the lab session of this afternoon.
- **Hint:** Create an array that records the number of incoming edges of all the vertices. Create a queue that keeps track of nodes with no incoming edges. When a node is given a number, update the array for all adjacent vertices. If an adjacent vertex now lost all its incoming edges then append it to the queue.

Motivation

Motivating Why We Want To Find Strongly Connected Components

- We now consider a classic application of depth-first search: decomposing a directed graph into its strongly connected components (SCCs).

Motivation

Motivating Why We Want To Find Strongly Connected Components

- We now consider a classic application of depth-first search: decomposing a directed graph into its strongly connected components (SCCs).
- Many algorithms that work with directed graphs begin with such a decomposition. After decomposing the graph into SCCs, such algorithms run separately on each one and then combine the solutions according to the structure of connections among SCCs.

Motivation

Motivating Why We Want To Find Strongly Connected Components

- We now consider a classic application of depth-first search: decomposing a directed graph into its strongly connected components (SCCs).
- Many algorithms that work with directed graphs begin with such a decomposition. After decomposing the graph into SCCs, such algorithms run separately on each one and then combine the solutions according to the structure of connections among SCCs.
- We will now see Kosaraju's algorithm to find SCCs that makes use of two depth-first searches.

Motivation

Motivating Why We Want To Find Strongly Connected Components

- We now consider a classic application of depth-first search: decomposing a directed graph into its strongly connected components (SCCs).
- Many algorithms that work with directed graphs begin with such a decomposition. After decomposing the graph into SCCs, such algorithms run separately on each one and then combine the solutions according to the structure of connections among SCCs.
- We will now see Kosaraju's algorithm to find SCCs that makes use of two depth-first searches.

Definition

Recall that a strongly connected component of a directed graph $G = (V, E)$ is a maximal set of vertices $C \subseteq V$ such that for every pair of vertices u and v in C , we have both $u \rightsquigarrow v$ and $v \rightsquigarrow u$, i.e., vertices u and v are reachable from each other.

Strongly Connected Components of a Directed Graph

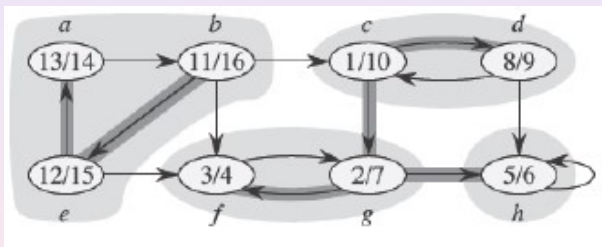


Figure: Each shaded region is a SCC of the given graph. The SCCs are: $\{a, b, e\}$, $\{f, g\}$, $\{c, d\}$, and $\{h\}$. Each vertex is labeled with its discovery and finishing times in a depth-first search, and the tree edges are shaded.

The Transpose Graph

G and G^T Have the Same SCCs

- Kosaraju's algorithm for finding SCCs of a graph $G = (V, E)$ uses the transpose of G , which is defined as $G^T = (V, E^T)$, where $E^T = \{(u, v) : (v, u) \in E\}$.

The Transpose Graph

G and G^T Have the Same SCCs

- Kosaraju's algorithm for finding SCCs of a graph $G = (V, E)$ uses the transpose of G , which is defined as $G^T = (V, E^T)$, where $E^T = \{(u, v) : (v, u) \in E\}$.
- Given an adjacency-list representation of G , the time to create G^T is $O(m + n)$.

The Transpose Graph

G and G^T Have the Same SCCs

- Kosaraju's algorithm for finding SCCs of a graph $G = (V, E)$ uses the transpose of G , which is defined as $G^T = (V, E^T)$, where $E^T = \{(u, v) : (v, u) \in E\}$.
- Given an adjacency-list representation of G , the time to create G^T is $O(m + n)$.
- Notice that G and G^T have exactly the SCCs because u and v are reachable from each other in G if and only if they are reachable from each other in G^T .

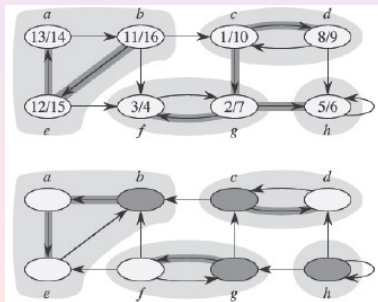


Figure: A directed graph and its transpose. The SCCs of both G and G^T are the same.

The Component Graph

Definition

For a directed graph $G = (V, E)$, we define its **component graph** $G^{SCC} = (V^{SCC}, E^{SCC})$ as follows:

- 1 Suppose that G has SCCs $C_1, C_2, \dots, C_k$. The vertex set V^{SCC} is $\{v_1, v_2, \dots, v_k\}$, and it contains a vertex v_i for each SCC C_i of G .
- 2 There is an edge $(v_i, v_j) \in E^{SCC}$ if G contains a directed edge (x, y) for some $x \in C_i$ and some $y \in C_j$.

The Component Graph

Definition

For a directed graph $G = (V, E)$, we define its **component graph** $G^{SCC} = (V^{SCC}, E^{SCC})$ as follows:

- 1 Suppose that G has SCCs $C_1, C_2, \dots, C_k$. The vertex set V^{SCC} is $\{v_1, v_2, \dots, v_k\}$, and it contains a vertex v_i for each SCC C_i of G .
- 2 There is an edge $(v_i, v_j) \in E^{SCC}$ if G contains a directed edge (x, y) for some $x \in C_i$ and some $y \in C_j$.
- Looked at another way, by contracting all edges whose incident vertices are within the same SCC of G , the resulting graph is G^{SCC} .

The Component Graph

Definition

For a directed graph $G = (V, E)$, we define its **component graph** $G^{SCC} = (V^{SCC}, E^{SCC})$ as follows:

- ❶ Suppose that G has SCCs $C_1, C_2, \dots, C_k$. The vertex set V^{SCC} is $\{v_1, v_2, \dots, v_k\}$, and it contains a vertex v_i for each SCC C_i of G .
- ❷ There is an edge $(v_i, v_j) \in E^{SCC}$ if G contains a directed edge (x, y) for some $x \in C_i$ and some $y \in C_j$.
 - Looked at another way, by contracting all edges whose incident vertices are within the same SCC of G , the resulting graph is G^{SCC} .
 - **Key property:** The component graph is a dag.

Properties of the Component Graph

Lemma

Let C and C' be distinct strongly connected components of a directed graph $G = (V, E)$. Let $u, v \in C$, let $u', v' \in C'$, and suppose that $u \rightsquigarrow u'$. Then, $v' \not\rightsquigarrow v$.

Properties of the Component Graph

Lemma

Let C and C' be distinct strongly connected components of a directed graph $G = (V, E)$. Let $u, v \in C$, let $u', v' \in C'$, and suppose that $u \rightsquigarrow u'$. Then, $v' \not\rightsquigarrow v$.

Proof

- If G contains a path $v' \rightsquigarrow v$, then it contains paths $u \rightsquigarrow u' \rightsquigarrow v'$ and $v' \rightsquigarrow v \rightsquigarrow u$.

Properties of the Component Graph

Lemma

Let C and C' be distinct strongly connected components of a directed graph $G = (V, E)$. Let $u, v \in C$, let $u', v' \in C'$, and suppose that $u \rightsquigarrow u'$. Then, $v' \not\rightsquigarrow v$.

Proof

- If G contains a path $v' \rightsquigarrow v$, then it contains paths $u \rightsquigarrow u' \rightsquigarrow v'$ and $v' \rightsquigarrow v \rightsquigarrow u$.
- Thus, u and v' are reachable from each other, thereby contradicting the assumption that C and C' are distinct SCCs.

Properties of the Component Graph

Lemma

Let C and C' be distinct strongly connected components of a directed graph $G = (V, E)$. Let $u, v \in C$, let $u', v' \in C'$, and suppose that $u \rightsquigarrow u'$. Then, $v' \not\rightsquigarrow v$.

Proof

- If G contains a path $v' \rightsquigarrow v$, then it contains paths $u \rightsquigarrow u' \rightsquigarrow v'$ and $v' \rightsquigarrow v \rightsquigarrow u$.
- Thus, u and v' are reachable from each other, thereby contradicting the assumption that C and C' are distinct SCCs.

Corollary

The component graph G^{SCC} of a directed graph G is a dag.

The Component Graph of a Directed Graph

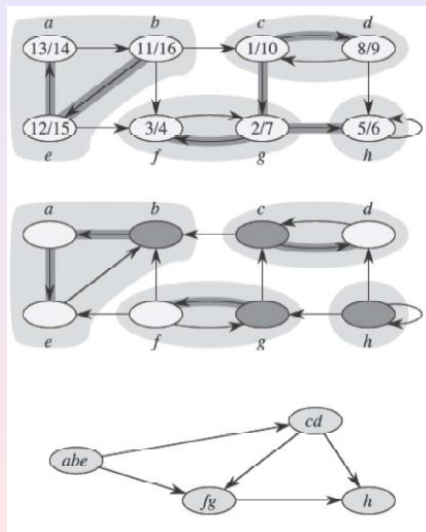


Figure: A directed graph G , its transpose G^T , and the component graph G^{SCC} .

Kosaraju's Algorithm

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 call DFS(G) to compute finishing times $u.f$ for each vertex u
- 2 compute G^T
- 3 call DFS(G^T), but in the main loop of DFS, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Figure: Pseudocode for the Kosaraju's Algorithm

A Sample Run of the Kosaraju's Algorithm

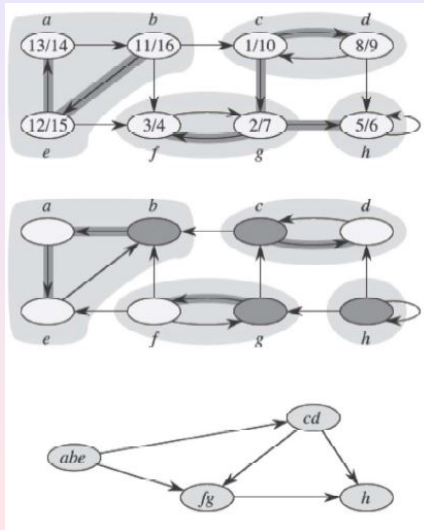


Figure: A sample run of the Kosaraju's Algorithm and the Component Graph

Key Observation and Notation

Key Observation

Key Observation: Considering vertices in the second DFS in decreasing order of the finishing times that were computed in the first DFS, we are, in essence, visiting the vertices of the component graph in topologically sorted order.

Key Observation and Notation

Key Observation

Key Observation: Considering vertices in the second DFS in decreasing order of the finishing times that were computed in the first DFS, we are, in essence, visiting the vertices of the component graph in topologically sorted order.

Resolving Potential Ambiguities and Introducing Notation

- Because Kosaraju's algorithm performs two DFS, there is the potential for ambiguity when we discuss $u.d$ or $u.f$. When discussing this algorithm, these values always refer to the discovery and finishing times as computed by the first call of DFS.

Key Observation and Notation

Key Observation

Key Observation: Considering vertices in the second DFS in decreasing order of the finishing times that were computed in the first DFS, we are, in essence, visiting the vertices of the component graph in topologically sorted order.

Resolving Potential Ambiguities and Introducing Notation

- Because Kosaraju's algorithm performs two DFS, there is the potential for ambiguity when we discuss $u.d$ or $u.f$. When discussing this algorithm, these values always refer to the discovery and finishing times as computed by the first call of DFS.
- We extend the notation for discovery and finishing times to sets of vertices.

Key Observation and Notation

Key Observation

Key Observation: Considering vertices in the second DFS in decreasing order of the finishing times that were computed in the first DFS, we are, in essence, visiting the vertices of the component graph in topologically sorted order.

Resolving Potential Ambiguities and Introducing Notation

- Because Kosaraju's algorithm performs two DFS, there is the potential for ambiguity when we discuss $u.d$ or $u.f$. When discussing this algorithm, these values always refer to the discovery and finishing times as computed by the first call of DFS.
- We extend the notation for discovery and finishing times to sets of vertices. If $U \subseteq V$, then we define $d(U) = \min_{u \in U} \{u.d\}$ and $f(U) = \max_{u \in U} \{u.f\}$.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.
- By the white-path theorem, all vertices in C and C' become descendants of x in the DFS tree. Since x has the latest finishing time of any of its descendants, $x.f = f(C) > f(C')$.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.
- By the white-path theorem, all vertices in C and C' become descendants of x in the DFS tree. Since x has the latest finishing time of any of its descendants, $x.f = f(C) > f(C')$.
- If $d(C) > d(C')$, let y be the first vertex discovered in C' . At time $y.d$, all vertices in C' are white and G contains a path from y to each vertex in C' consisting only of white vertices.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.
- By the white-path theorem, all vertices in C and C' become descendants of x in the DFS tree. Since x has the latest finishing time of any of its descendants, $x.f = f(C) > f(C')$.
- If $d(C) > d(C')$, let y be the first vertex discovered in C' . At time $y.d$, all vertices in C' are white and G contains a path from y to each vertex in C' consisting only of white vertices.
- By the white-path theorem, all vertices in C' become descendants of y in the DFS tree, and thus $y.f = f(C')$.

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.
- By the white-path theorem, all vertices in C and C' become descendants of x in the DFS tree. Since x has the latest finishing time of any of its descendants, $x.f = f(C) > f(C')$.
- If $d(C) > d(C')$, let y be the first vertex discovered in C' . At time $y.d$, all vertices in C' are white and G contains a path from y to each vertex in C' consisting only of white vertices.
- By the white-path theorem, all vertices in C' become descendants of y in the DFS tree, and thus $y.f = f(C')$.
- At time $y.d$, all vertices in C are white. But since there is an edge (u, v) from C to C' , there cannot be a path from C' to C .

Proving The Observation

Lemma

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E$, where $u \in C$ and $v \in C'$. Then $f(C) > f(C')$.

Proof

- We consider two cases, depending on which SCC had the first discovered vertex.
- If $d(C) < d(C')$, let x be the first vertex discovered in C . At time $x.d$, all vertices in C and C' are white, and so x has a path to each vertex in C and C' consisting of white vertices.
- By the white-path theorem, all vertices in C and C' become descendants of x in the DFS tree. Since x has the latest finishing time of any of its descendants, $x.f = f(C) > f(C')$.
- If $d(C) > d(C')$, let y be the first vertex discovered in C' . At time $y.d$, all vertices in C' are white and G contains a path from y to each vertex in C' consisting only of white vertices.
- By the white-path theorem, all vertices in C' become descendants of y in the DFS tree, and thus $y.f = f(C')$.
- At time $y.d$, all vertices in C are white. But since there is an edge (u, v) from C to C' , there cannot be a path from C' to C .
- Hence, no vertex in C is reachable from y . At time $y.f$, therefore, all vertices in C are still white. Thus, for any vertex $w \in C$, we have $w.f > y.f$, which implies that $f(C) > f(C')$.

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .
- We start with the SCC C whose finishing time $f(C)$ is maximum. The search starts from some vertex $x \in C$, and it visits all vertices in C .

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .
- We start with the SCC C whose finishing time $f(C)$ is maximum. The search starts from some vertex $x \in C$, and it visits all vertices in C .
- By this corollary, G^T contains no edges from C to any other SCC, and so the search from x will not visit vertices in any other SCC. Thus, the tree rooted at x contains exactly the vertices of C .

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .
- We start with the SCC C whose finishing time $f(C)$ is maximum. The search starts from some vertex $x \in C$, and it visits all vertices in C .
- By this corollary, G^T contains no edges from C to any other SCC, and so the search from x will not visit vertices in any other SCC. Thus, the tree rooted at x contains exactly the vertices of C .
- Having completed visiting all vertices in C , the second DFS selects as a root a vertex from some other SCC C' whose finishing time $f(C')$ is maximum over all SCCs other than C .

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .
- We start with the SCC C whose finishing time $f(C)$ is maximum. The search starts from some vertex $x \in C$, and it visits all vertices in C .
- By this corollary, G^T contains no edges from C to any other SCC, and so the search from x will not visit vertices in any other SCC. Thus, the tree rooted at x contains exactly the vertices of C .
- Having completed visiting all vertices in C , the second DFS selects as a root a vertex from some other SCC C' whose finishing time $f(C')$ is maximum over all SCCs other than C .
- Again, the search will visit all vertices in C' , but by the corollary, the only edges in G^T from C' to any other SCC must be to C , which we have already visited.

Proving Correctness of Kosaraju's Algorithm

Corollary

Let C and C' be distinct SCCs of a directed graph $G = (V, E)$. Suppose that there is an edge $(u, v) \in E^T$, where $u \in C$ and $v \in C'$. Then $f(C) < f(C')$.

Proving Algorithm by Using the Corollary

- This corollary provides the key to understanding why the Kosaraju's algorithm works.
- Let us examine what happens when we perform the second DFS, which is on G^T .
- We start with the SCC C whose finishing time $f(C)$ is maximum. The search starts from some vertex $x \in C$, and it visits all vertices in C .
- By this corollary, G^T contains no edges from C to any other SCC, and so the search from x will not visit vertices in any other SCC. Thus, the tree rooted at x contains exactly the vertices of C .
- Having completed visiting all vertices in C , the second DFS selects as a root a vertex from some other SCC C' whose finishing time $f(C')$ is maximum over all SCCs other than C .
- Again, the search will visit all vertices in C' , but by the corollary, the only edges in G^T from C' to any other SCC must be to C , which we have already visited.
- In general, when the DFS of G^T visits any SCC, any edges out of that SCC must be to an SCC that the search already visited. Each DFS tree, therefore, will be exactly one SCC.